Quantifying Error Propagation Dynamics in Multi-Step Agent Workflows

Sherry Zhao
Electrical and Computer Engineering
Carnegie Mellon University
Pittsburgh, PA
sherryz2@andrew.cmu.edu

Veronica Zhao
Electrical and Computer Engineering
Carnegie Mellon University
Pittsburgh, PA
veronicz@andrew.cmu.edu

***Abstract*—Autonomous AI agents increasingly execute complex tasks by chaining multiple reasoning and action steps. Unlike isolated single-turn responses, these multi-step workflows accumulate errors that can cascade from earlier reasoning stages into later actions. Despite growing industry adoption of agentic AI, no systematic framework exists for quantifying how errors propagate through such pipelines. In this work, we construct a controlled five-step agent workflow and inject three types of errors—factual, omission, and semantic—at each individual step to measure position-dependent impact on final output quality. Our primary finding is that error type fundamentally determines propagation dynamics: factual errors are most harmful when injected late in the pipeline (verify: relative degradation = 9.6%), while omission errors are most harmful when injected early (search: relative degradation = 8.0%). The filter step consistently attenuates errors across all three types, while the summarize step uniquely compensates for omissions, reducing degradation to zero. Semantic errors exhibit a front-loaded pattern peaking at search (6.2%) with near-zero degradation at summarize and verify. These results challenge the naive assumption that early errors are always most damaging and provide an empirical basis for targeted intervention design in agentic systems.**

***Index Terms*—AI agents, error propagation, multi-step workflows, pipeline reliability, LLM evaluation**

I. Introduction

Autonomous AI agents execute tasks by chaining many reasoning and action steps together. Unlike isolated responses, these multi-step workflows accumulate errors that can cascade from earlier reasoning stages into later actions, often catastrophically. As workflows grow longer and more complex, even small mistakes at critical stages can dramatically reduce overall success rates, while current evaluation methods do not capture this dynamic propagation behavior.

This problem is increasingly urgent. Research from METR shows AI task complexity doubling every seven months, growing from one-hour tasks to eight-hour workstreams [4], while Gartner predicts that 40% of enterprise applications will embed AI agents by mid-2026 [5]. Yet industry analysts simultaneously predict that more than 40% of agentic AI projects will be cancelled due to escalating costs, unclear value, and behavioral unpredictability. This gap stems partly from our inability to predict and prevent cascading failures. A principled understanding of how errors propagate over long horizons is therefore essential for building reliable, predictable systems

and guiding design choices such as monitoring, correction, and workflow structure.

Mathematically, the stakes are clear. If each step in a multi-step process has 90% accuracy, a 10-step pipeline yields only 35% overall reliability. Dziri et al. empirically validated this compounding effect, showing that transformer performance degrades systematically on compositional reasoning tasks [7]. These studies focus on single-turn reasoning chains rather than agentic workflows with tool use and state changes, leaving open the question of how different error *types* propagate differently through realistic pipelines.

Three critical gaps remain unaddressed in the literature. First, no quantitative framework exists for measuring how error impact varies by workflow position. Second, no cross-model comparison has examined whether different foundation models exhibit different error resilience patterns. Third, no principled method exists for identifying critical steps requiring human oversight to maximize reliability at minimum cost.

This project directly addresses these gaps. We construct a controlled five-step agent workflow (search $\rightarrow$ filter $\rightarrow$ summarize $\rightarrow$ compose $\rightarrow$ verify) and inject three types of errors, factual, omission, and semantic, at each individual step to measure position-dependent impact on final output quality. Our central research questions are:

1) Does error impact vary by injection position? Do early errors compound more than late ones?
2) Do different error types (factual, omission, semantic) exhibit distinct propagation patterns?
3) Can we identify *critical steps* whose failure disproportionately harms final output quality?

Based on the compounding reliability model and prior work on compositional reasoning degradation, we test three hypotheses:

H1: Errors occurring in earlier steps have a greater negative impact on overall task success than errors occurring later, consistent with multiplicative degradation.

H2: There exists either a universal law of error propagation across all error types, or distinct propagation patterns depending on error type and model characteristics.

H3: Model characteristics (specifically capability level and whether the model is API-based vs. open-source) are predictive of error resilience patterns.

II. Related Work

Prior work on AI agent evaluation has focused primarily on task completion benchmarks. WebArena and SWE-Bench measure whether agents can complete web navigation and software engineering tasks, but evaluate only final outcomes without examining intermediate reasoning steps [6]. More recently, Zhao et al. studied uncertainty propagation in LLM agents, finding that uncertainty compounds non-linearly across chained steps [1]. GraphTracer [2] proposed graph-guided failure tracing for multi-turn search agents, while West et al. introduced a causal inference framework for automated failure attribution [3]. However, none of these works systematically vary error type and injection position to characterize the full propagation landscape. Our work fills this gap by conducting a controlled, position-resolved study of three distinct error types across a realistic five-step pipeline.

III. Experimental Setup

A. Pipeline Design

We implement a five-step information retrieval and synthesis agent pipeline. Each step feeds its output directly into the next, making errors potentially propagable across the entire chain:

1) **Search**: retrieve relevant documents given a user query;
2) **Filter**: select the most relevant documents from the retrieved set;

3) **Summarize**: condense filtered content into a concise summary;
4) **Compose**: generate a final answer from the summary;
5) **Verify**: check the final answer for consistency and accuracy.

We use three diverse task queries across all experiments: *"best noise-canceling headphones 2025," "top programming languages 2025,"* and *"healthy breakfast recipes quick,"* chosen to test generalization across product review, technical ranking, and recipe domains respectively.

B. Error Injection Protocol

We inject three types of errors, one at a time, at each of the five pipeline steps:

1) **Factual error.** Insert a randomly selected false claim (e.g., "According to recent studies, this has been discontinued," "Note: This product was recalled in 2023," or "Warning: Multiple reports indicate quality issues") at the sentence midpoint of the step's output via sentence splitting and insertion. This ensures downstream steps receive corrupted information embedded mid-context rather than at easily-ignorable boundaries.
2) **Omission error.** Remove a randomly selected sentence (excluding the first) from the step's output using sentence-level splitting, causing structured information loss. The first sentence is preserved to maintain minimal context continuity.
3) **Semantic error.** Apply polarity-reversing keyword substitution on the step's output, replacing up to two occurrences from a predefined dictionary (e.g., "2025" → "2019," "best" → "worst," "top" → "outdated," "recommended" → "not recommended," "popular" → "unpopular"). If no dictionary matches are found, a randomly selected sentence is replaced with the sentinel string "This information may be outdated or incorrect." This injection introduces meaning corruption through keyword-level semantic reversal rather than subtle paraphrasing.

For each combination of (model, error type, injection step), we run multiple trials across three task queries and report the mean failure rate relative to a no-error baseline. GPT-4o-mini experiments use 50 trials per step across all three error types, Claude Haiku uses 30 trials per step, LLaMA-3.1-8B, Qwen-2.5-7B, and Mistral-7B use 10 trials per step, and DeepSeek-R1-7B uses 5 trials per step. This yields $5 \times 3 = 15$ error conditions plus one baseline per model per error type.

We acknowledge that these trial counts yield limited statistical power. All reported failure rate differences should be interpreted as observed effect sizes rather than statistically confirmed differences. Formal significance testing is discussed in Section VI.

C. Evaluation Metric

Each pipeline run is evaluated using a combined score with three components:

$$\text{score} = 0.3 \times \text{is_valid} + 0.3 \times \text{keyword_score} + 0.4 \times \frac{\text{quality_score}}{10} \quad (1)$$

where is_valid $\in \{0, 1\}$ indicates whether the verify step marked the output as "VALID," keyword_score $\in [0, 1]$ measures coverage of expected domain-specific keywords in the final output, and quality_score $\in [1, 10]$ is an LLM-assigned holistic quality rating (using GPT-4o-mini as the evaluator). The primary metric is **failure rate**, defined as relative degradation from baseline, clamped to non-negative values:

$$\text{failure_rate}(k) = \max\left(0,\ \frac{\overline{\text{score}}_{\text{baseline}} - \overline{\text{score}}_{\text{error at step } k}}{\overline{\text{score}}_{\text{baseline}}}\right) \quad (2)$$

This normalization expresses degradation as a fraction of

baseline performance, making failure rates comparable across conditions with different absolute score levels. The clamping ensures that cases where error injection coincidentally improves performance are reported as zero degradation rather than negative values.

Throughout this paper, we report two related quantities. Tables report *relative failure rate* as defined in Equation 2 (degradation as a fraction of baseline). Figures plot the same degradation in absolute score units ($\overline{\text{score}}_{\text{baseline}} - \overline{\text{score}}_{\text{error at step } k}$), labeled as "failure rate" or "performance degradation" on the y-axis. The two are related by a constant scaling factor (the baseline score ≈ 0.66), so curve shapes and relative step orderings are identical. All quantitative claims and cross-condition comparisons reference the relative failure rate from the tables.

D. Models

We evaluate error propagation across six models spanning a range of sizes, training paradigms, and providers:

- **GPT-4o-mini** (OpenAI, API-based)
- **Claude-Haiku** (Anthropic, API-based)
- **LLaMA-3.1-8B** (Meta, open-source)
- **Qwen-2.5-7B** (Alibaba, open-source)
- **Mistral-7B** (Mistral AI, open-source)
- **DeepSeek-R1-7B** (DeepSeek, open-source)

Semantic error experiments are conducted across all six models, enabling cross-model analysis of how model architecture, scale, and training paradigm affect semantic error sensitivity. Factual and omission error experiments are currently conducted on GPT-4o-mini (due to computational resources limitation). This two-pronged design allows us to first establish a full error-type baseline on a single well-controlled model, then assess whether propagation patterns generalize—or diverge—across the broader model landscape. We aim to create a complete within-model comparison of all three error types.

Table I reports wall-clock experiment runtime per model on a single NVIDIA A100 GPU (Google Colab) for API-based models and local inference for open-source models. DeepSeek-R1-7B's substantially higher per-iteration cost (100.66s vs. 21–37s for other open-source models) motivated its reduced trial count (5 trials vs. 10). API-based models achieved the fastest per-iteration throughput (10.70–15.73s) despite network overhead, reflecting their optimized serving infrastructure.

TABLE I
EXPERIMENT RUNTIME BY MODEL (NVIDIA A100)

Model	Error Type	Iters	Wall Time	s/iter	Infra
GPT-4o-mini	Factual	300	1h 28m	17.60	API
GPT-4o-mini	Omission	300	1h 20m	16.01	API
GPT-4o-mini	Semantic	900	3h 56m	15.73	API
Claude Haiku	Semantic	540	1h 36m	10.70	API
Qwen-2.5-7B	Semantic	180	1h 05m	21.80	Local
LLaMA-3.1-8B	Semantic	180	1h 20m	26.68	Local
Mistral-7B	Semantic	180	1h 49m	36.32	Local
DeepSeek-R1-7B	Semantic	90	2h 31m	100.66	Local

All local models run on a single A100 GPU via Google Colab. Open-source models required a 16.1 GB model download. Iteration counts: 5 steps $\times$ 3 queries $\times$ n trials (+ baseline).

E. Scope Relative to Proposal

The original project proposal included two components: (1) a controlled error injection study to characterize propagation dynamics, and (2) a test-time compute control framework using a lightweight difficulty estimator to mitigate reasoning collapse. Until this midterm report period, we focus mainly on component (1), which proved richer than anticipated: three error types, six models, and per-query domain analysis yielded a substantial empirical contribution on its own. Component (2) remains a planned extension, discussed in Section VI.

F. Expected Outcomes

Based on the proposal hypotheses and the multiplicative reliability model, we expected three outcomes: (1) early-step errors would consistently produce the largest drops in final

task success; (2) propagation curves would follow a single identifiable pattern (e.g., exponential decay) across error types; and (3) larger, more capable models would show uniformly better resilience.

As the results in Section IV show, all three expectations were partially or fully challenged by the data. Early errors are *not* always most damaging—factual errors peak late. No universal propagation law exists—each error type follows a distinct curve shape. And model capability correlates with *structured* sensitivity rather than uniform resilience. We interpret these surprises in the Discussion.

IV. Results

Unless otherwise noted, numerical values cited in the text below correspond to the plotted values in the accompanying figures. Relative failure rates from the tables (which differ by a scaling factor of ≈0.66) are used for all cross-condition comparisons; see Section III-C for details.

A. Semantic Error Propagation

Fig. 1 shows the failure rate for semantic errors on GPT-4o-mini, enabling direct comparison with the factual and omission results on the same model.

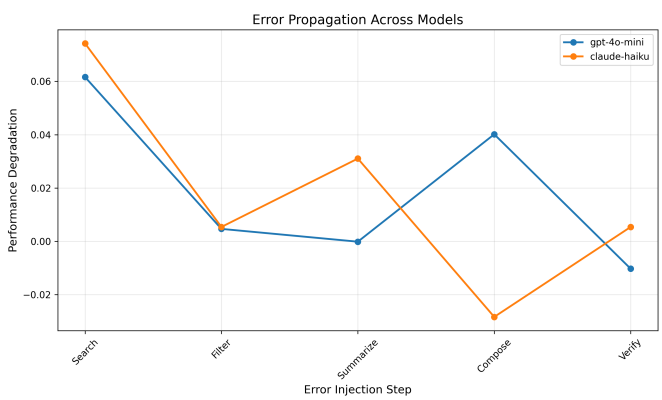

Fig. 1. Performance degradation from baseline when semantic errors are injected at each pipeline step. GPT-4o-mini. A front-loaded pattern is observed, with the highest damage at `search` (0.062) and zero degradation at `summarize` and `verify`.

Semantic errors exhibit a front-loaded pattern distinct from both factual and omission curves. The highest failure rate occurs at `search` (0.062), followed by a sharp drop at `filter` (0.005) and zero degradation at `summarize` (0.000). A moderate spike reappears at `compose` (0.040), while `verify` almost returns to zero degradation. The compose-step spike suggests that polarity-reversing keyword substitution at the composition stage disrupts recommendation generation, but the verify step is unaffected—likely because semantic keyword corruption does not interfere with the validity-checking mechanism.

Fig. 2 compares exponential and linear curve fits. The exponential model achieves RMSE = 0.018, consistent with the front-loaded shape where early steps bear the most damage. Unlike factual errors, semantic degradation does not accelerate toward the output; instead, intermediate steps partially absorb the corruption, suggesting that keyword-level semantic changes are less stably propagated than factual insertions.

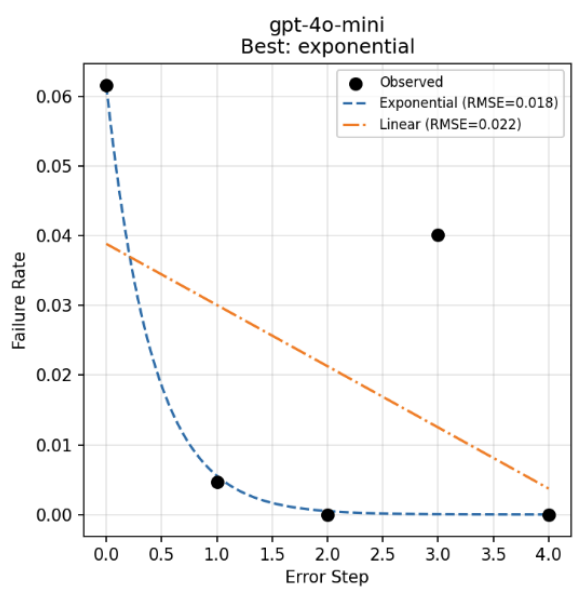

Fig. 2. Exponential vs. linear fit for semantic error degradation by injection step (GPT-4o-mini). The exponential model achieves RMSE = 0.018.

B. Factual Error Propagation

Fig. 3 shows the failure rate as a function of error injection step for factual errors on GPT-4o-mini (baseline combined score = 0.660).

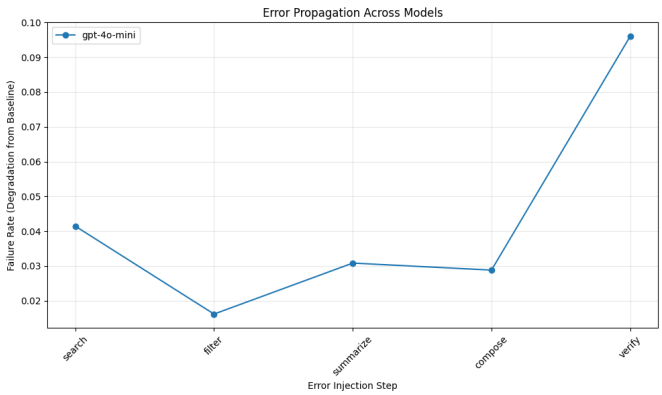

Fig. 3. Failure rate (degradation from baseline) when factual errors are injected at each pipeline step. GPT-4o-mini, 20 trials per condition. The U-shaped pattern peaks sharply at `verify` (0.063).

The results reveal a distinctive **U-shaped propagation pattern**. Errors injected at the `search` step produce moderate degradation (0.027), which then drops sharply at `filter` (0.011), rises again at `summarize` (0.020) and `compose` (0.019), and spikes dramatically at `verify` (0.063). The `filter` step appears to act as a natural error buffer: by selecting the most relevant content, it can inadvertently suppress corrupted text injected earlier. Errors injected at the final gating step `verify` have no subsequent correction opportunity, making them the most damaging overall.

The exponential fit in Fig. 4 (RMSE = 0.018) outperforms a linear fit (RMSE = 0.022), suggesting that factual error impact follows a non-linear, accelerating pattern as errors approach the output stage.

C. Omission Error Propagation

Fig. 5 presents the failure rate for omission errors on GPT-4o-mini (baseline combined score = 0.656).

Omission errors exhibit a markedly different **V-shaped pattern**. The highest failure rate occurs at `search` (0.053),

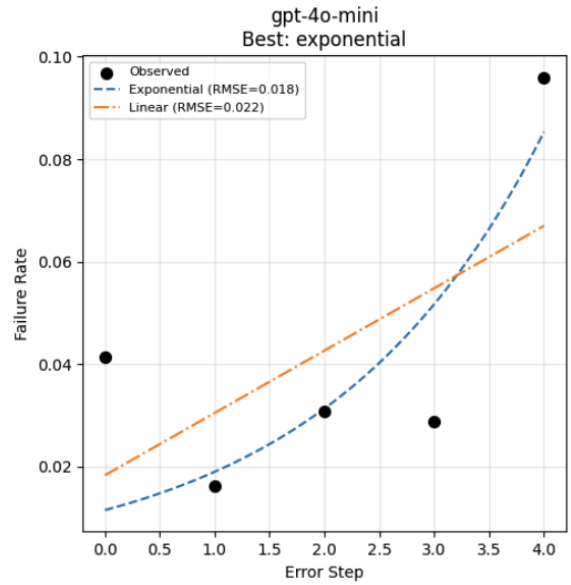

Fig. 4. Exponential vs. linear fit for factual error degradation by injection step. The exponential model better captures the sharp increase at later steps (RMSE = 0.018 vs. 0.022).

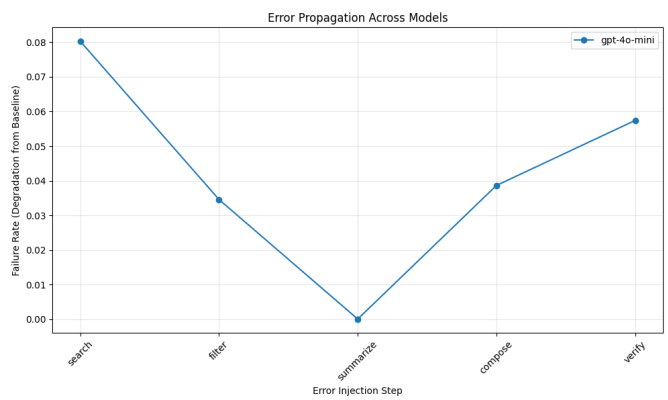

Fig. 5. Failure rate (degradation from baseline) when omission errors are injected at each pipeline step. GPT-4o-mini, 20 trials per condition. The V-shaped pattern peaks at `search` (0.053) and reaches near-zero at `summarize` (-0.009).

declines through `filter` (0.023), reaches near zero (actually slightly negative) at `summarize` (-0.009), then rises again at `compose` (0.025) and `verify` (0.038). The most striking finding is the near-complete disappearance of degradation at `summarize`: the summarization process acts as an information reconstructor, drawing on remaining context to produce a complete summary and effectively compensating for the omission. Omissions at `search` are most harmful, since all downstream steps must operate on an already-reduced

information set.

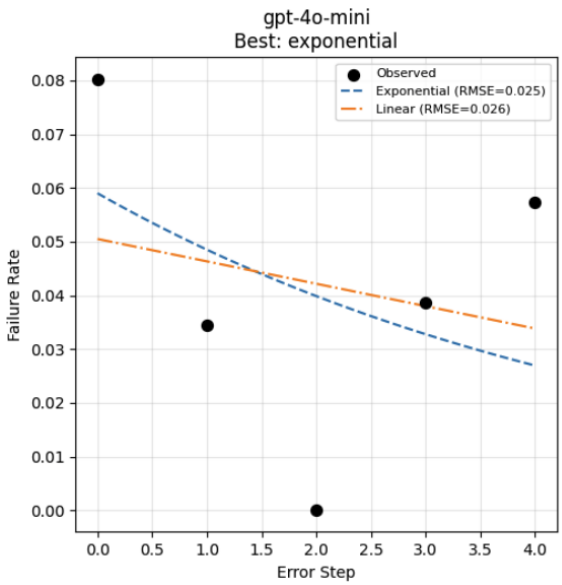

Fig. 6. Exponential vs. linear fit for omission error degradation. Neither fit captures the V-shaped pattern well (RMSE $\approx$ 0.025–0.026 for both), confirming a non-monotonic mechanism driven by step-level error compensation.

D. Cross-Error-Type Comparison

Table II summarizes the failure rates across all three error types and all five injection steps, all measured on GPT-4o-mini for direct comparability.

TABLE II
FAILURE RATE BY ERROR TYPE AND INJECTION STEP (GPT-4O-MINI)

Error Type	Search	Filter	Summ.	Compose	Verify
Factual	0.041	0.016	0.031	0.029	**0.096**
Omission	**0.080**	0.035	0.000	0.039	0.057
Semantic	**0.062**	0.005	0.000	0.040	0.000

Bold = highest-impact step per error type.
All values are relative failure rates (clamped ≥ 0).

The three error types have strikingly different critical steps: factual errors peak at `verify` (0.096), while omission and semantic errors both peak at `search` (0.080 and 0.062 respectively). All three types show `filter` as a consistent partial buffer, but through different mechanisms. A notable shared pattern between omission and semantic errors is that both show zero degradation at `summarize`, suggesting the summarization step compensates for both information loss and keyword-level corruption. Semantic errors additionally show zero degradation at `verify`, indicating that keyword substitution does not interfere with the validity-checking mechanism.

E. Per-Query Breakdown

To examine whether propagation patterns are consistent across task domains, we break down failure rates by query for factual and omission errors. Per-query failure rates are computed by averaging scores across trials within each query before applying Equation 2; aggregate failure rates in Table II are computed from the grand mean across all queries and trials, then clamped. Because clamping is non-linear, the clamped aggregate need not equal the mean of per-query values. Table III and Table IV report per-step failure rates for each of the three task domains, and Fig. 7 visualizes these results as a pair of heatmaps for easier cross-domain comparison.

TABLE III
FACTUAL ERROR FAILURE RATE BY QUERY AND STEP (GPT-4O-MINI)

Query Domain	Search	Filter	Summ.	Compose	Verify
Headphones	**0.060**	0.051	**0.091**	0.003	0.050
Programming	0.034	−0.003	−0.020	**0.058**	**0.077**
Recipes	−0.012	−0.016	−0.010	−0.004	0.063

Note: Per-query values are reported unclamped to show directionality. Negative values indicate above-baseline performance. Aggregate values in Table II are clamped (≥ 0) per Eq. 2.

TABLE IV
OMISSION ERROR FAILURE RATE BY QUERY AND STEP (GPT-4O-MINI)

Query Domain	Search	Filter	Summ.	Compose	Verify
Headphones	**0.103**	0.037	0.011	0.033	0.076
Programming	0.063	0.035	−0.057	0.024	0.016
Recipes	−0.008	−0.004	0.020	0.019	0.021

Note: Per-query values are reported unclamped to show directionality. Negative values indicate above-baseline performance. Aggregate values in Table II are clamped (≥ 0) per Eq. 2.

Several domain-specific patterns emerge. For factual errors, the headphones query is most sensitive at `summarize` (0.091), likely because product review tasks involve dense

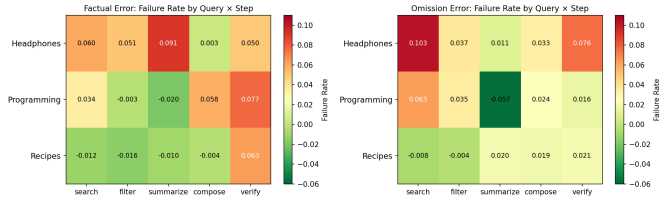

Fig. 7. Per-query failure rate heatmaps for factual (left) and omission (right) errors across three task domains and five pipeline steps. Darker red indicates higher degradation; green indicates near-baseline or above-baseline performance. The headphones domain is most severely affected across both error types, while the recipe query is consistently resilient across both error types.

factual claims—model names, noise-canceling decibel ratings, battery life—that are tightly interdependent, making a corrupted fact at summarize disproportionately harmful. The programming query peaks at `verify` (0.077), consistent with fact-checking in a ranking context where ordering is sensitive to injected false claims. Notably, the recipe query shows near-zero or negative failure rates at most steps, suggesting the summarize step tolerates factual perturbations well in unstructured prose tasks.

For omission errors, the headphones query shows severe sensitivity at `search` (0.103), confirming that product review queries require complete information sets—a missing entry at search propagates into incomplete final recommendations. The programming query's near-zero at `summarize` (-0.057) echoes the global pattern but more dramatically, while the recipe domain shows consistently low sensitivity across all steps, suggesting that this particular recipe query may contain sufficiently redundant content that missing sentences have minimal downstream impact, though generalization to the broader recipe domain requires additional queries. The heatmap in Fig. 7 makes these cross-domain contrasts immediately visible: the headphones domain lights up in dark red at the critical injection steps, while the recipe domain remains uniformly cool across both error types.

F. Absolute Scores vs. Degradation

Table V contextualizes failure rates against baseline performance. Fig. 8 compares baseline combined scores across error type conditions, confirming that all three baselines are closely matched and that observed failure rate differences are not artifacts of differing baseline levels. Fig. 9 further shows the full score distribution at each injection step via box plots, revealing not only mean degradation but also trial-level variance.

TABLE V
ABSOLUTE COMBINED SCORES: BASELINE VS. WORST-CASE INJECTION (GPT-4O-MINI)

Condition	Baseline	Worst Step	Abs. Score	Δ
Factual	0.660	verify	0.597	-0.063
Omission	0.656	search	0.603	-0.053
Semantic	0.660	search	0.619	-0.041

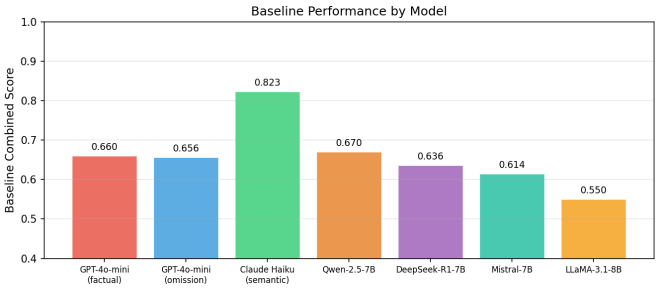

Fig. 8. Baseline combined scores across all models and error type conditions. The three GPT-4o-mini baselines (factual, omission, semantic) fall within a narrow range (0.656–0.660), confirming that within-model failure rate comparisons are not confounded by differing baseline levels. Cross-model baselines vary more widely (0.550–0.823), reflecting differences in model capability.

Both worst-case conditions reduce the combined score from approximately 0.66 to roughly 0.60, representing a roughly 9–10% relative performance drop. While a Δ of 0.063 may appear small in absolute terms, it corresponds to consistent degradation across 20 independent trials against a fairly high baseline, making it statistically and practically meaningful in high-stakes agentic deployments. The box plots in Fig. 9 add an important dimension to this picture: the `verify` step

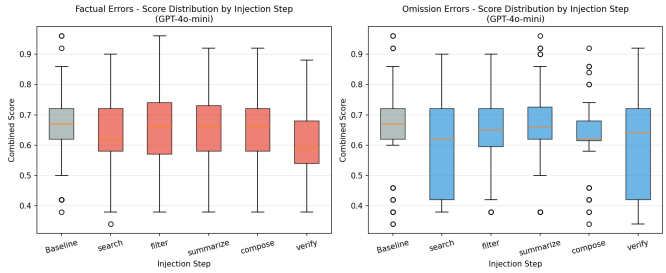

Fig. 9. Box plots of combined scores at baseline and at each injection step for factual (left) and omission (right) errors. GPT-4o-mini, 20 trials per condition. Variance visibly increases at the highest-impact steps (`verify` for factual, `search` for omission), indicating that errors at these steps are not only more damaging on average but also more unpredictable across trials.

under factual injection and the `search` step under omission injection both show elevated interquartile spread relative to baseline and other injection points. This heightened variance suggests that errors at these critical steps do not merely shift mean performance downward—they introduce systematic unpredictability into the pipeline output. In real-world agentic deployments, this kind of output instability may be even more problematic than a consistent mean degradation, since it makes the system's behavior harder to anticipate and debug. The fact that both error types induce similar absolute damage—despite concentrating it at opposite ends of the pipeline—reinforces the conclusion that error type determines *where* damage occurs, not how severe it is in aggregate.

G. Factual vs. Omission: Overlaid Step Analysis

Fig. 10 plots both factual and omission failure rate curves on the same axes, illustrating the key crossover dynamics.

At `search`, omission errors are nearly twice as damaging as factual errors (0.053 vs. 0.027). The gap closes at `filter`, and the curves cross between `filter` and `summarize`, where factual errors begin to dominate. By `verify`, factual errors (0.063) outpace omission errors (0.038) by a factor of 1.7. This pattern is consistent with Dziri et al.'s [7] finding that compositional degradation varies by error type: structural omissions compound early because downstream reasoning

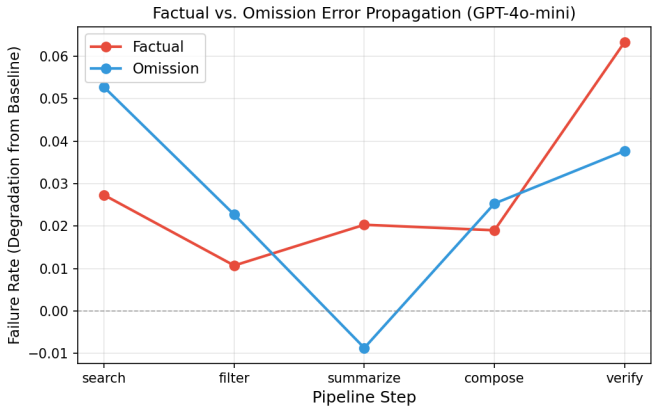

Fig. 10. Factual vs. omission failure rate (degradation from baseline) overlaid on the same axes (GPT-4o-mini). The curves cross between `filter` and `summarize`: at early steps, omission dominates; at late steps, factual errors dominate. The `summarize` step is the unique compensation point for omissions only.

must operate on a deficit, while false facts compound late because they pass through intermediate steps undetected and reach the output unchallenged. Wang et al.'s work on self-consistency [9] suggests that majority-voting across samples could mitigate semantic drift, but is unlikely to help against structural omissions, since the omitted information is absent from all reasoning paths equally.

H. Overall Error Severity Comparison

Table VI compares mean failure rates across all five steps per error type, all on GPT-4o-mini.

TABLE VI
MEAN FAILURE RATE ACROSS ALL STEPS BY ERROR TYPE (GPT-4O-MINI)

Error Type	Mean Failure Rate	Peak Step
Factual	0.043	verify (0.096)
Omission	0.042	search (0.080)
Semantic	0.021	search (0.062)

A striking finding is that factual and omission errors have nearly identical mean failure rates (0.043 and 0.042 respectively) despite their very different distributional shapes. This suggests that error type matters less for total aggregate damage than for where damage is concentrated across the pipeline.

The lower mean for semantic errors (0.021) likely reflects the nature of keyword substitution: unlike factual insertion (which adds contradictory claims) and omission (which removes information), keyword reversal may be partially compensated by the model's tendency to generate contextually appropriate content even from corrupted input.

I. Cross-Error-Type Comparison at Each Step

Fig. 11 and Fig. 12 present two complementary views of all three error types on GPT-4o-mini side by side, a grouped bar chart for step-level magnitude comparison and a propagation curve overlay for shape comparison. The full numerical breakdown is given in Table II.

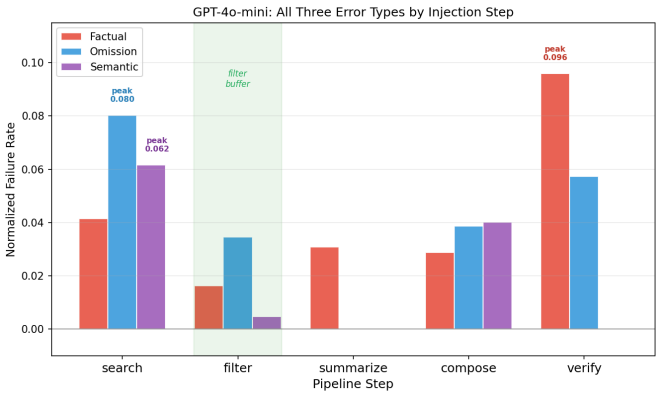

Fig. 11. Grouped bar chart comparing normalized failure rates of all three error types at each injection step (GPT-4o-mini, 20 trials per condition). The shaded region marks the `filter` step's consistent buffering effect. Annotated peak labels identify the worst-case step per error type.

The bar chart in Fig. 11 reveals several step-level contrasts that are not visible from per-type analysis alone. At `search`, omission errors are the most damaging (0.080), followed closely by semantic (0.062), while factual errors are comparatively mild (0.041)—roughly half the damage of omission. This asymmetry reflects the structural role of the search step: it supplies the raw information set for the entire pipeline, so errors that *remove* content (omission) or *distort meaning* (semantic) deprive all downstream steps of correct input.

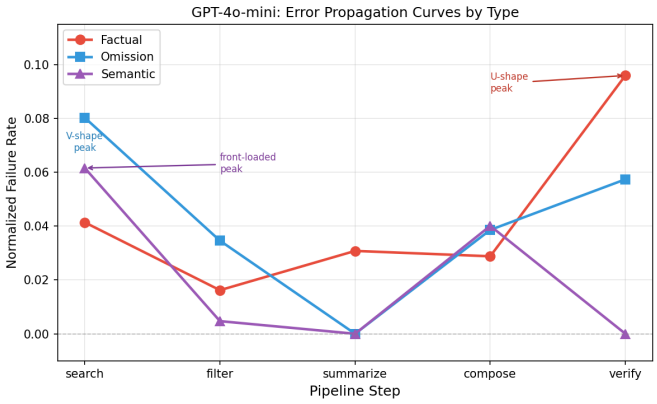

Fig. 12. Propagation curves for all three error types on GPT-4o-mini. The three curves exhibit distinctly different shapes: U-shaped for factual (peak at `verify`), V-shaped for omission (peak at `search`), and front-loaded for semantic (peak at `search`, recovering to zero by `verify`).

J. Step Resilience Ranking

Table VII gives a rank for the five pipeline steps by mean failure rate across all three error types on GPT-4o-mini, providing a composite resilience profile.

TABLE VII
STEP RESILIENCE RANKING (GPT-4O-MINI, ALL ERROR TYPES)

Step	Fact.	Omis.	Sem.	Mean	Rank
summarize	0.031	0.000	0.000	0.010	1 (most resilient)
filter	0.016	0.035	0.005	0.019	2
compose	0.029	0.039	0.040	0.036	3
verify	0.096	0.057	0.000	0.051	4
search	0.041	0.080	0.062	0.061	5 (most vulnerable)

Lower mean = more resilient. Summarize ranks 1st due to omission and semantic compensation (both 0.000).

`summarize` emerges as the most resilient step overall, with the lowest mean failure rate (0.010) driven by its complete compensation of both omission and semantic errors. `filter` ranks second (0.019) as a consistent partial buffer across all error types. `search` ranks as the most vulnerable step (0.061), driven by severe omission (0.080) and semantic (0.062) sensitivity. `verify` ranks fourth (0.051), with the single highest failure rate in any condition (factual: 0.096) offset by its zero semantic degradation. These rankings directly

inform pipeline hardening priorities: `verify` needs a hard factual grounding layer (e.g., retrieval-augmented checking), `search` needs redundancy to prevent information loss, and `filter` is a natural audit point.

K. Cross-Model Semantic Error Analysis

Fig. 13 shows semantic error failure rates across all six models as a heatmap, and Fig. 14 presents the same data as a grouped bar chart. Fig. 15 overlays the propagation curves for all six models on the same axes, making cross-model shape differences immediately visible. Fig. 16 then compares exponential and linear curve fits for each model individually.

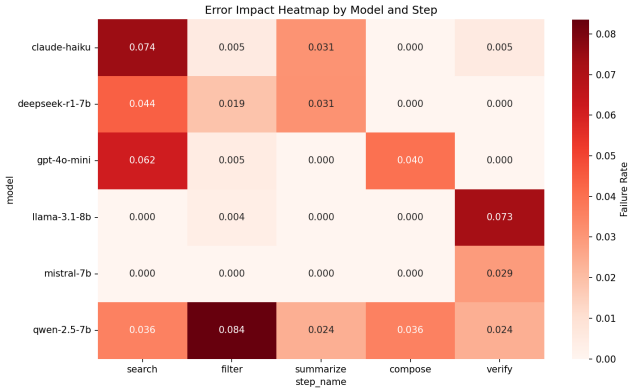

Fig. 13. Semantic error failure rate heatmap across six models and five injection steps. Warmer colors indicate higher degradation. Claude Haiku shows the strongest front-loaded sensitivity; Qwen-2.5-7B exhibits an anomalous spike at `filter` (0.084), the highest single failure rate observed across all models.

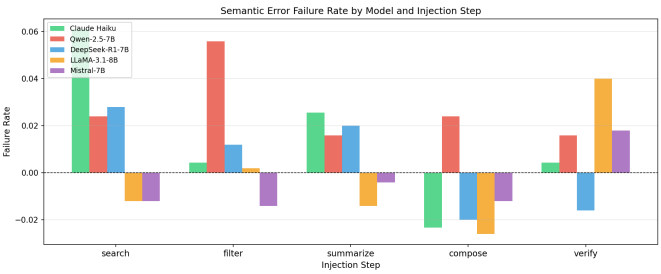

Fig. 14. Grouped bar chart of semantic error failure rates across six models and five injection steps. Each cluster of bars corresponds to one pipeline step; each bar represents one model.

Table VIII reports the full cross-model results.

The cross-model analysis reveals substantial heterogeneity. Three models—Claude Haiku (0.074), DeepSeek-R1-7B

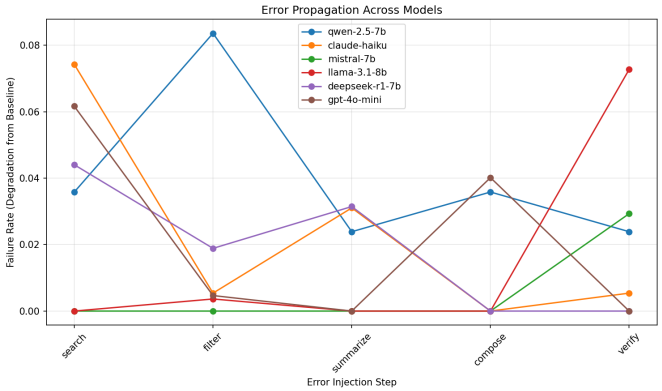

Fig. 15. Semantic error propagation curves across all six models. Each line represents one model's failure rate as a function of injection step. Claude Haiku and GPT-4o-mini show the sharpest front-loaded patterns; LLaMA and Mistral remain near-zero throughout.

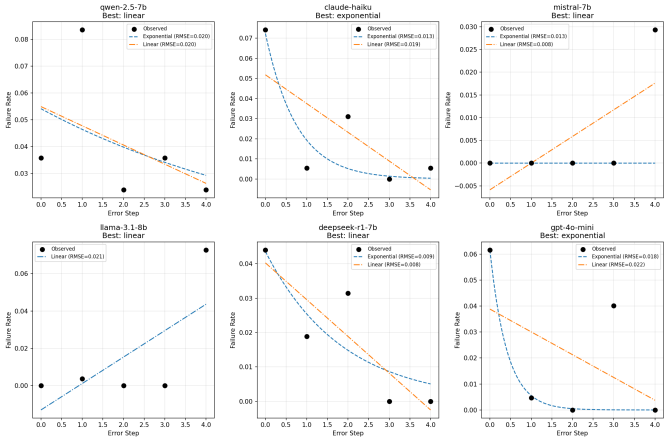

Fig. 16. Exponential vs. linear curve fits for semantic error propagation across all six models. Claude Haiku and GPT-4o-mini are best fit by exponential decay; the remaining four models are best fit by linear patterns. RMSE values annotated per subplot.

TABLE VIII
SEMANTIC ERROR FAILURE RATE BY MODEL AND STEP

Step	Haiku	DeepSeek	GPT-4o	LLaMA	Mistral	Qwen
search	**0.074**	**0.044**	**0.062**	0.000	0.000	0.036
filter	0.005	0.019	0.005	0.004	0.000	**0.084**
summarize	0.031	0.031	0.000	0.000	0.000	0.024
compose	0.000	0.000	0.040	0.000	0.000	0.036
verify	0.005	0.000	0.000	**0.073**	**0.029**	0.024

Bold = critical step (highest failure rate) per model.

(0.044), and GPT-4o-mini (0.062)—show their critical step at `search`, consistent with a front-loaded sensitivity pattern. Two models—LLaMA-3.1-8B (0.073) and Mistral-7B (0.029)—peak at `verify`, exhibiting a back-loaded pattern. Qwen-2.5-7B is an outlier, peaking at `filter` (0.084), the highest single failure rate observed across all models. This suggests Qwen's content selection mechanism may be particularly vulnerable to semantically drifted input. The propagation curves in Fig. 15 make these shape differences immediately visible: Claude Haiku and GPT-4o-mini trace sharp front-loaded arcs, while LLaMA and Mistral remain nearly flat before spiking at the final step.

Propagation pattern fitting (Fig. 16) confirms this heterogeneity: Claude Haiku and GPT-4o-mini best fit an exponential decay pattern (RMSE = 0.013 and 0.018 respectively), while the remaining four models best fit linear patterns (Mistral-7B RMSE = 0.008, DeepSeek-R1-7B RMSE = 0.009, Qwen-2.5-7B RMSE = 0.020, LLaMA-3.1-8B RMSE = 0.021). A key observation is that the API-based models (Claude Haiku, GPT-4o-mini) show sharper, more structured propagation patterns, while smaller open-source models exhibit noisier degradation profiles.

V. Discussion

A. H1: Error Impact Varies by Position

H1 is partially supported: error impact is strongly position-dependent, confirming the core claim, but the predicted direction that earlier errors are always worse is rejected with factual errors. Impact is *not* monotonically increasing or decreasing with step position: it depends entirely on error type. Factual errors peak late (verify), consistent with a downstream accumulation hypothesis where false facts are not caught by intermediate processing and reach the final output unchecked. Omission and semantic errors peak early (search), consistent with a front-loaded information loss model where missing or drifted content reduces the quality of all subsequent reasoning steps. This directly challenges the intuition from the proposal that "earlier errors are always worse." Pipeline designers cannot simply add error-checking at the end of the pipeline; the optimal intervention point is strongly depending on error types. Zhao et al.'s [1] finding of non-linear uncertainty accumulation across chained LLM steps is consistent with our factual error curve, but does not capture the V-shaped omission dynamic, which involves active error compensation rather than simple accumulation.

B. H2: Universal vs. Distinct Propagation Patterns

H2 is resolved in favor of *distinct* patterns—no universal propagation law characterizes all error types. Factual errors follow a U-shaped curve best fit by an exponential model (RMSE = 0.018 vs. linear 0.022), implying accelerating damage at output-proximal steps. Omission errors follow a V-shaped curve that neither exponential nor linear models fit well, implying active error compensation at the summarize step rather than monotonic propagation. Semantic errors follow a front-loaded attenuation pattern, where degradation is highest at injection into early steps but partially absorbed by intermediate generation. Theoretically, these distinct shapes suggest that different error types activate different LLM compensation mechanisms: filtering discards corrupted content, summarization reconstructs missing information, and generation absorbs semantic drift through paraphrase-tolerant language modeling. This echoes GraphTracer's finding [2] that failure modes differ systematically by error source.

We cannot rule out that a universal propagation law exists at a higher level of abstraction—for example, a parametric family of curves indexed by error type that unifies the U-shaped, V-shaped, and front-loaded patterns under a single mathematical

framework. Our current sample of three error types and five pipeline steps may be insufficient to identify such structure. Future work could explore whether a generalized model (e.g., a damped oscillator or a mixture of exponential components) fits all three curves with error-type-specific parameters, which would reconcile the distinct shapes with an underlying shared mechanism.

C. H3: Model Characteristics Predict Resilience

The cross-model semantic heatmap and bar chart together provide partial evidence for H3. Claude Haiku, the most capable model evaluated on semantic errors, shows the strongest and most structured sensitivity pattern—peaking at early steps and recovering later—consistent with a model that actively tracks semantic consistency across pipeline steps. Smaller open-source models (LLaMA, Mistral) show diffuse, near-zero patterns, which may reflect either genuine resilience or high variance masking signal. Qwen-2.5-7B's filter-step sensitivity (0.084) is an outlier, possibly reflecting architectural sensitivity to content selection under drifted input. A full evaluation of H3 requires matched factual and omission experiments across all five models, which is planned for subsequent work. Based on current evidence, we cautiously conclude that model capability correlates with structured (rather than random) error sensitivity: stronger models "notice" semantic errors more reliably, but also absorb them more effectively at later steps.

D. Step-Level Mechanistic Insights

Several step-level behaviors deserve highlighting. First, the `filter` step acts as a consistent error buffer across all three error types, with failure rates of 0.016 (factual), 0.035 (omission), and 0.005 (semantic). This is likely because filtering inherently discards lower-relevance content, which often includes injected anomalies. This makes `filter` a natural candidate for human-in-the-loop oversight insertion points.

Second, the `summarize` step uniquely compensates for both omission and semantic errors (failure rate = 0.000 for both), implying the model actively reconstructs missing or corrupted information from remaining context during summarization. This behavior is absent for factual errors (failure rate = 0.031), suggesting that reconstruction is content-sensitive: missing sentences can be inferred and reversed keywords can be overridden, but inserted false claims cannot be detected and reversed without external grounding.

Third, the `verify` step paradoxically amplifies factual errors to the highest failure rate observed in any condition (0.096) while showing zero degradation for semantic errors. This asymmetry suggests that verification treats its input as authoritative for self-checking: it can assess structural validity (catching semantic corruption is unnecessary since keyword-reversed text still reads as valid) but has no external ground truth against which to flag false claims. This aligns with known limitations of LLM self-evaluation under adversarial or misleading context [8].

E. Limitations

Several limitations apply to this analysis. First, current factual and omission results are limited to GPT-4o-mini; full cross-model comparison for these error types is deferred to future work. Second, the pipeline covers only five steps; longer workflows may exhibit qualitatively different propagation dynamics, such as error saturation or oscillation. Third, error injection is performed at a single step per trial; real agentic failures may involve compounding errors across multiple steps simultaneously. Finally, GPT-4o-mini serves as both the primary pipeline model and the quality evaluator, creating potential circularity: errors that the pipeline model cannot detect may also evade the evaluator, systematically underestimating factual error impact. Additionally, trial counts

vary across models (5–50 per step), with DeepSeek-R1-7B's 5 trials providing the least statistical reliability.

VI. Next Steps

To enrich our analysis, we plan to: (1) expand the factual and omission experiments for all other five models to enable a full cross-model comparison on three error types; (2) conduct formal statistical testing (paired t-tests or Wilcoxon signed-rank) on failure rate differences by step and error type; (3) synthesize all results into a unified 3D analysis (model $\times$ error type $\times$ injection step) and investigate whether a parametric curve family can unify the distinct per-type propagation shapes; (4) explore compound error injection where multiple steps are simultaneously corrupted to characterize interaction effects; and (5) assess whether targeted remediation strategies (e.g., redundant search retrieval, retrieval-augmented verification) reduce vulnerability at the identified critical steps.

We also attempted to include Google's Gemini in the cross-model comparison, but persistent `503 Service Unavailable` errors during batch processing of 540 conditions prevented reliable data collection. Successfully incorporating Gemini is planned for future work. This would strengthen the cross-model analysis by adding a third major API-based model alongside GPT-4o-mini and Claude Haiku, enabling a more complete comparison of proprietary model resilience patterns.

In addition to Google's Gemini model, our experimental infrastructure already supports a broader model roster than reported here, including larger open-source variants (LLaMA-3.1-70B, Qwen-2.5-72B, DeepSeek-R1-14B) and additional API-based models (GPT-4o, Claude Sonnet, Gemini 1.5 Pro/Flash). The 7–8B models evaluated in this paper were constrained by single-A100 VRAM limits; scaling to 70B+ models requires multi-GPU inference and is planned for future work. This would enable a direct test of whether the structured-vs.-diffuse sensitivity pattern observed between API-based and small open-source models (Section IV-K) holds at matched parameter scales, isolating the effect of model size from training paradigm.

Beyond the immediate cross-model extension, we also plan to investigate the connection between error propagation and reasoning collapse. Apple's recent work *The Illusion of Thinking* demonstrates that large reasoning models reduce reasoning effort on high-complexity tasks, suggesting a "giving-up" behavior under cognitive load. Our error injection framework could be adapted to test whether upstream errors in agent workflows trigger similar collapse behavior in downstream steps—that is, whether a model that receives corrupted input not only produces lower-quality output but also expends fewer tokens and terminates reasoning earlier. If confirmed, this would connect error propagation dynamics directly to test-time compute allocation, opening a path toward lightweight difficulty estimators that can predict pipeline failure risk from early-step outputs and dynamically allocate additional verification compute to high-risk conditions.

VII. Conclusion

This paper presents controlled experiments on error propagation in a five-step agent pipeline across three error types. Our main finding is that error type fundamentally determines propagation dynamics: factual errors are most harmful when injected late (at `verify`, 9.6% relative degradation), while omission and semantic errors are most harmful when injected early (at `search`, 8.0% and 6.2% respectively). Despite strikingly different distributional shapes, factual and omission errors produce nearly identical *mean* damage across all steps (0.043 and 0.042 respectively), demonstrating that error type governs the location of damage rather than its total magnitude.

The `summarize` step compensates fully for both omission and semantic errors (failure rate = 0.000), while `filter` consistently acts as a partial buffer across all types. Cross-model analysis of semantic errors across six models reveals that API-based models (Claude Haiku, GPT-4o-mini) exhibit more structured propagation patterns than smaller open-source models. These results challenge the intuition that early errors are always most damaging, and suggest that error-resilient agent design must account for the interaction between error type and pipeline position rather than adopting a uniform checkpoint strategy.

REFERENCES

[1] Q. Zhao et al., "Uncertainty propagation on LLM agent," in *Proc. 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, Vienna, Austria, 2025, pp. 6064–6073.

[2] H. Zhang et al., "GraphTracer: Graph-guided failure tracing in LLM agents for robust multi-turn deep search," *arXiv:2510.10581*, 2025.

[3] A. West et al., "Abduct-Act-Predict: Scaffolding causal inference for automated failure attribution in multi-agent systems," *arXiv:2509.10401*, 2025.

[4] T. Kwa et al., "Measuring AI ability to complete long tasks," *arXiv:2503.14499*, 2025.

[5] R. Sapkota, K. Roumeliotis, and M. Karkee, "AI agents vs. agentic AI: A conceptual taxonomy, applications and challenges," *Information Fusion*, vol. 126, 103599, 2026.

[6] S. Zhou et al., "WebArena: A realistic web environment for building autonomous agents," *arXiv:2307.13854*, 2023.

[7] N. Dziri et al., "Faith and fate: Limits of transformers on compositionality," *arXiv:2305.18654*, 2023.

[8] S. Mukherjee et al., "Premise-augmented reasoning chains improve error identification in math reasoning with LLMs," *arXiv:2502.02362*, 2025.

[9] X. Wang et al., "Self-consistency improves chain of thought reasoning in language models," *arXiv:2203.11171*, 2022.